GERAÇÃO DE BATIDAS BOOM BAP COM UMA GRAMÁTICA PROBABILÍSTICA LIVRE DE CONTEXTO

Guilherme Novais de Souza
Departamento de Ciência da Computação, Universidade Federal de Minas Gerais

RESUMO

Este trabalho apresenta um gerador de batidas de boom bap, o hip-hop dos anos 1990, baseado só na expansão de uma gramática probabilística livre de contexto (PCFG): uma parte estrutural indexada deriva a forma da música com o número exato de compassos, e uma tabela de regras define o estilo. A amostragem é reprodutível pela semente e monotônica nos hiperparâmetros, e em 100 sementes sorteadas nenhuma música violou as restrições de estilo.

1. INTRODUÇÃO

Gramáticas formais são uma representação natural para músicas com forma hierárquica e convencionada [1, 2]. Uma batida de hip-hop é uma árvore: a música se divide em seções, as seções em blocos de 4 ou 8 compassos, e os compassos em eventos de percussão numa grade. O boom bap tem groove padronizado (bumbo no primeiro tempo, caixa nos tempos 2 e 4, swing de MPC), mas varia dentro do padrão. Este trabalho descreve o `boombap`, que gera músicas completas nesse estilo, em MIDI, só pela expansão de uma PCFG em que (i) forma e duração são garantidas por construção, (ii) as restrições de estilo são verificadas sobre a tabela de regras antes da geração e (iii) a amostragem é reprodutível e monotônica nos hiperparâmetros.

2. TRABALHOS RELACIONADOS

Roads e Wieneke [1] fazem um panorama do uso de gramáticas como representação musical. O trabalho mais próximo deste é o Bol Processor de Kippen e Bel [3], em que gramáticas formais representam sequências de toques de tabla: como aqui, os terminais são eventos percussivos e a gramática codifica um estilo específico. Keller e Morrison [4] usam uma gramática probabilística para improvisar jazz, com terminais abstratos que só depois viram notas, separação parecida com a nossa entre derivação e realização.

3. MÉTODO

3.1 Restrição de Estilo e Representação

O estilo alvo é o boom bap dos anos 1990: compasso 4/4, 85–95 bpm, swing de MPC, forma intro → verso → refrão → verso → outro, com cada seção composta de blocos de 4 ou 8 compassos. Cada compasso é uma grade de 16 semicolcheias (posições 0–15, com 120 ticks cada e 480 ticks por semínima). Um terminal de bateria tem a forma `peça@posição:velocity`, por exemplo `kick@0:118`, mapeado para o canal 10 do General MIDI. Os outros terminais são marcadores: `grid8`/`grid16` dá a grade do chimbal (e quais posições recebem swing), `dv`$\pm$`k` e `dt`$\pm$`k` são desvios de velocity e de tempo, ⟨`seção:X`⟩ marca uma seção, e `deg`k é um grau da escala menor natural, tocado pelo baixo a cada ataque de bumbo, com a tônica na oitava 2.

3.2 A Gramática

A gramática é $G = (N, \Sigma, R, S)$ com uma função $P : R \to [0, 1]$ tal que $\sum_\alpha P(A \to \alpha) = 1$ para todo $A \in N$. O axioma é $S = \textit{Música}(n)$, com n o número de compassos pedido. Para $n = 40$, a listagem do programa tem 193 não-terminais, 100 terminais e 386 regras. Destes, 121 têm duas ou mais produções com $0 < P < 1$, de modo que a gramática é não-determinística: o mesmo axioma gera músicas diferentes.

Parte estrutural. As famílias da Figura 1 (acima) são geradas pelo código, mas todos os seus pesos vêm de parâmetros da tabela de regras. Os índices são o número de compassos k, o groove g e um estado s. Toda regra conserva a soma dos índices de duração do lado direito, logo qualquer derivação de $\textit{Música}(n)$ tem exatamente n compassos, sem amostragem por rejeição. O groove é sorteado uma vez por seção, e o estado s registra o que precede o compasso (por exemplo, um prato após uma virada). Para cada n os índices tomam valores finitos; instanciá-los dá um conjunto finito de não-terminais comuns, e a gramática continua livre de contexto.

Parte de estilo. A tabela `boombap.rules` define três grooves (Classic, Lazy e Bounce), com bumbos e caixas fixos, notas opcionais e um padrão de chimbal; viradas *Fill_g*, que trocam o final do compasso por uma de cinco "caudas" (rufo, bumbo–caixa etc.); e a humanização: cada terminal de bateria x equivale a $X \to x\ \textit{HumV}\ \textit{HumT}$. Com o baixo ligado, cada bumbo também expande *BassFirst* ou *BassDeg*, que sorteiam o grau.

Restrições validadas. Antes de cada geração, a tabela é verificada em toda a grade de `density` e `fills`: probabilidades somam 1; todo groove tem bumbo no 0, caixa no 4 e no 12 e um bumbo sincopado; toda virada mantém bumbo no 0 e caixa no 4; todo compasso tem chimbal e

© G. N. de Souza. Trabalho prático 1 de DCC831 – Inteligência Artificial na Música, DCC/UFMG, 2026. Relatório no formato ISMIR.

Música(n)	$\to$ *Forma*(n)	[1]
Forma(n)	$\to$ *Intro*(i) *Corpo*$(n{-}i{-}o)$ *Outro*(o)	$[\propto P_I(i)\, P_O(o)]$
Corpo(m)	$\to$ *Verso*(a) *Refrão*(b) *Verso*(c)	$[\propto w(a,b,c)]$
Verso(k)	$\to$ ⟨`seção:verso`⟩ *Seção*$(k, g,$ início_seção$)$	$[P_G(g)]$
Seção(k, g, s)	$\to$ *Bloco4*(g, s) *FimBloco*$(k{-}4, g)$	[0,4]
	| *Bloco8*(g, s) *FimBloco*$(k{-}8, g)$	[0,6]
Bloco4(g, s)	$\to$ *Compasso*$(g,$ primeiro$, s)$ *Compasso*$(g,$ meio, normal$)^2$	[1]
FimBloco(k, g)	$\to$ *Compasso*$(g,$ fim, virada$)$ *Seção*$(k, g,$ pós_virada$)$	[@fill_end $= 0{,}30 + 0{,}70f$]
	| *Compasso*$(g,$ fim, groove$)$ *Seção*$(k, g,$ normal$)$	[resto]
Compasso$(g, \cdot, s)$	$\to$ *Prato*(s) *Fill_g* | *Prato*(s) *Groove_g*	[@fill_mid $= 0{,}05 + 0{,}25f$ | resto]
Prato(s)	$\to$ `crash@0:110` $\mid \varepsilon$	[p_s | resto]
Groove_Classic	$\to$ `kick@0:118 snare@4:115 kick@10:104 snare@12:117` *OptKicks_Classic Ghosts_Classic Hats_Classic*	[1]
OptKick7_Classic	$\to$ `kick@7:88` $\mid \varepsilon$	[$0{,}20 + 0{,}55d$ | resto]
Hats_Classic	$\to$ *Hats16* | *Hats8_Classic*	[$0{,}10 + 0{,}50d$ | resto]

Figura 1. Acima, as famílias estruturais indexadas; abaixo, algumas regras da tabela de estilo `boombap.rules`. d e f são `density` e `fills`. Em *Forma*, $i, o \in \{4, 8\}$; em *Corpo*, a, b, c são múltiplos de 4 com $a{+}b{+}c = m$ e $w = \exp(-\|(a,b,c)/m - (0{,}3;\ 0{,}4;\ 0{,}3)\|^2/2\sigma^2)$, $\sigma = 0{,}08$; $g \in \{\text{Classic, Lazy, Bounce}\}$ com $P_G = (0{,}45;\ 0{,}25;\ 0{,}30)$. *Intro*, *Refrão* e *Outro* são análogas a *Verso*; *Seção*$(0, g, s) \to \varepsilon$.

uma grade; `@fill_end` > `@fill_mid`; e não há recursão infinita. Senão, a tabela é recusada.

3.3 Realização: dos Terminais ao MIDI

Um evento na posição p do compasso c toca no tick $1920c + 120p + \delta(p) + dt$, limitado ao compasso. No swing da MPC, com w%, as posições fracas da grade atrasam $\delta = (w/100 - 1/2) \cdot 480$ ticks nas colcheias (metade nas semicolcheias): $w = 50$ é reto e $w \approx 67$ é ternário. `dv` soma-se à velocity. O MIDI guarda a semente e os hiperparâmetros, e a derivação completa vai para um `.json`.

Hiperparâmetros. `density` e `fills` (0–1, padrão 0,5) controlam notas opcionais e viradas e entram como expressões nas probabilidades da tabela, como $0{,}20 + 0{,}55\,d$ em *OptKick7_Classic*. Os demais são `swing` (52–75%), `bpm` (85–95), `bars` (20–256, múltiplo de 4), `bass`/`key` (baixo em tom menor) e a semente.

3.4 Reprodutibilidade

O número $u \in [0, 1)$ de cada nó vem do hash sha256(*semente* | *caminho*) do seu caminho na árvore, e a regra é escolhida pela CDF inversa na ordem da tabela. A mesma semente gera arquivos idênticos, e mudar um parâmetro não muda nenhum u. Como as probabilidades são não-decrescentes no parâmetro e a alternativa "presente" vem primeiro, $u < p(d_1) \Rightarrow u < p(d_2)$ para $d_1 \le d_2$: toda nota opcional presente com `density` baixa continua presente com `density` alta.

4. RESULTADOS

Geramos quatro amostras (Tabela 1), cada uma com `.mid`, `.json` e `.wav` (FluidSynth): `seed7_default` (padrões), `seed7_busy` (mesma semente, $d = f = 1$), `seed21_bass_dm` (outra semente, swing 70 e baixo em Ré menor) e `seed99_long` (86 bpm, 64 compassos, $d = 0{,}2$, $f = 0{,}3$). Os arquivos estão em `samples/` no repositório do projeto [1].

[1] `https://github.com/guilhermenovais/boombap-generation`

Amostra	bumbo	caixa	aro	chimbal	viradas	atraso
`seed7_default`	2,52	2,48	0,55	8,55	10/40	33,1
`seed7_busy`	2,77	2,75	0,75	8,85	17/40	31,1
`seed21_bass_dm`	2,42	2,88	0,28	9,55	10/40	74,6
`seed99_long`	2,50	2,56	0,14	8,86	13/64	33,6
seed 7, $d = 0$	2,35	2,33	0,28	7,30	10/40	37,8
seed 7, $d = 1$	2,83	2,52	0,85	9,43	10/40	30,3

Tabela 1. Saída de `boombap analyze`: notas por compasso (chimbal fechado), compassos com virada e atraso médio das posições fracas, em ticks (120 = uma semicolcheia).

Entre `seed7_default` e `seed7_busy` a forma é a mesma (4–12–12–8–4), porque nenhuma regra estrutural depende de `density`. As viradas passam de 10 para 17 compassos e os bumbos, caixas e aros aumentam. Variando só `density` (duas últimas linhas), o número de viradas fica igual e bumbo, aro e chimbal crescem. Com swing 70, `seed21_bass_dm` tem mais que o dobro do atraso das demais.

Validação. Os testes geram 100 sementes com hiperparâmetros sorteados e analisam cada MIDI: nenhuma violou as restrições de estilo, e as seções somam o número de compassos pedido. Regenerar com os parâmetros gravados reproduz o MIDI byte a byte. A monotonicidade em `density` vale nas 10 sementes testadas, mas em 3 delas `density` 0→1 multiplicou bumbo+chimbal por menos que a meta de 1,3 (1,27 a 1,29): os coeficientes de `density` poderiam ser maiores.

5. DISCUSSÃO E TRABALHOS FUTUROS

O sistema gera só bateria e baixo, sem melodia, harmonia ou samples. As probabilidades foram escritas à mão, e, por ser livre de contexto, a gramática não repete motivos entre seções; também não houve teste com ouvintes. Como trabalhos futuros, as probabilidades poderiam ser estimadas do Groove MIDI Dataset [5] e gramáticas sensíveis ao contexto permitiriam repetir motivos.

6. REFERÊNCIAS

[1] C. Roads and P. Wieneke, "Grammars as representations for music," *Computer Music Journal*, vol. 3, no. 1, pp. 48–55, 1979.

[2] F. Lerdahl and R. Jackendoff, *A Generative Theory of Tonal Music*. Cambridge, MA: MIT Press, 1983.

[3] J. Kippen and B. Bel, "Modelling music with grammars: Formal language representation in the Bol Processor," in *Computer Representations and Models in Music*, A. Marsden and A. Pople, Eds. London: Academic Press, 1992, pp. 207–238.

[4] R. M. Keller and D. R. Morrison, "A grammatical approach to automatic improvisation," in *Proc. of the 4th Sound and Music Computing Conf. (SMC)*, Lefkada, Greece, 2007.

[5] J. Gillick, A. Roberts, J. Engel, D. Eck, and D. Bamman, "Learning to groove with inverse sequence transformations," in *Proc. of the 36th Int. Conf. on Machine Learning (ICML)*, ser. PMLR, vol. 97, 2019, pp. 2269–2279.